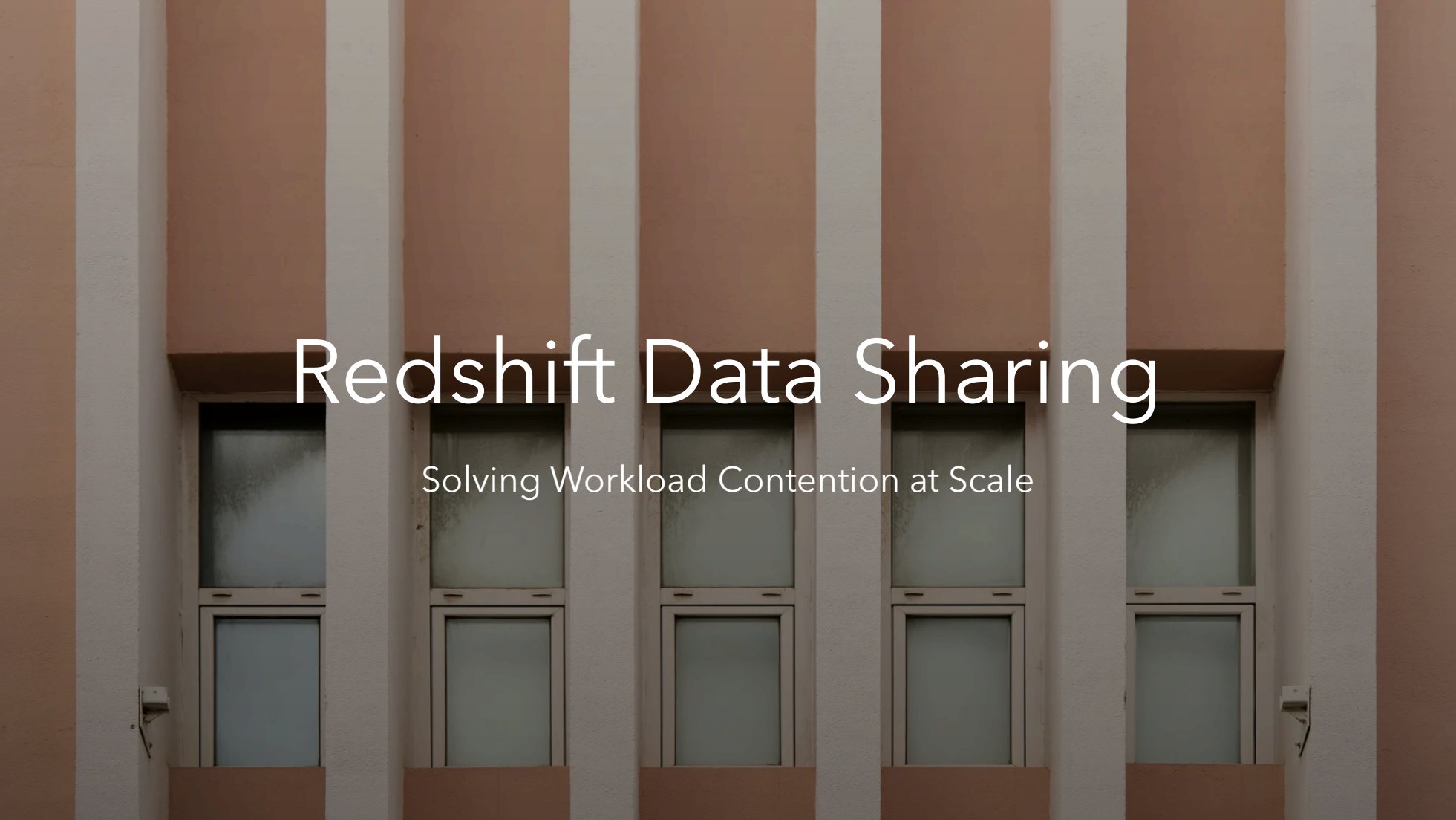


Redshift Data Sharing

Solving Workload Contention at Scale

Data sharing in Redshift

> "Data sharing lets you share live data, without having to create a copy or move it."
> Source: AWS Redshift Developer Guide – Data sharing in Amazon Redshift

Data sharing in Redshift

```
+———————————————————+           +———————————————————+
| Producer Cluster  |           | Consumer Cluster  |
|                   |           |                   |
| Local tables/views |          | CREATE DATABASE ...|
|                   |           | FROM DATASHARE    |
+————————+————————+           +————————+————————+
         |                                |
         | creates                        | references
         v                                |
     +———————————————————+                |
     | Datashare: Sales Data |<————————+
     | Thin metadata layer   |
     | No copied table data  |
     +————————+————————+
              |
              | points to
              v
     +———————————————————+
     | Redshift Managed Storage |
     | S3-backed live data      |
     +———————————————————+
```

- `Sales Data` is the thin sharing layer, not a copied dataset

Setup – Producer Cluster

```sql
CREATE DATASHARE shared_sales_data
WITH (WRITE_ACCESS = TRUE);

-- aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee
SELECT current_namespace;

GRANT USAGE ON DATASHARE shared_sales_data
TO NAMESPACE '12345678-1234-1234-1234-123456789012';

ALTER DATASHARE shared_sales_data
ADD SCHEMA sales;

ALTER DATASHARE shared_sales_data
ADD ALL TABLES IN SCHEMA sales;

ALTER DATASHARE shared_sales_data
SET INCLUDENEW = TRUE FOR SCHEMA sales;
```

Setup – Consumer Cluster

```sql
CREATE DATABASE shared_sales_data
FROM DATASHARE shared_sales_data
OF NAMESPACE 'aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee';

CREATE EXTERNAL SCHEMA sales
FROM REDSHIFT DATABASE 'shared_sales_data' SCHEMA shared_sales_data
```

Setup - Additional Resources

- Amazon Redshift Data Sharing: Setup Guide 2024
  Step-by-step walkthrough covering datashare creation, permissions, consumer setup, monitoring, and troubleshooting.

- YouTube video walkthrough
  Supplemental video resource for seeing the Redshift data sharing flow explained visually.

Why bother with Data Sharing?

Isolation, burst compute, testing, and redundancy

The Problem - One Cluster Rules Them All

```
        ETL                 Audit
         \                    /
          \                  /
           v                v
+———————————————————————————————————+
|     Amazon Redshift Cluster       |
|                                   |
|   +————————+    +————————+        |
|   | Leader  ├————┤ Compute |       |
|   |  Node   |    |  Nodes  |       |
|   +————————+    +————————+        |
|                                   |
+———————————————————————————————————+
          ^                ^
         /                  \
        /                    \
Marketing                 Finance
```

- One shared Redshift cluster often becomes the center of the entire data platform
- Very different teams and workloads all converge on the same compute at the same time
- This is the contention problem the rest of the presentation will unpack

Stepping Stone - Workload Management

- Control concurrency
- Allocate memory
- Prioritize workloads
- Evicts lower priority queries when needed

```
ETL ———————— Priority 1 ---\
Audit ————— Priority 1 ———\
Marketing ——— Priority 2 ———+——> [ WLM ] ——> +——————————————+
Finance ——— Priority 3 ———/                  | Redshift Cluster |
                                             +——————————————+
```

WLM Types

Manual

- Static queues
- Fixed allocation

Automatic

- ML-based
- Dynamic memory + slots

Stepping Stone - Workload Management

- You hit a classic **chicken and egg** problem:
  - ETL pipelines need steady throughput
  - Client-facing queries need low latency
- Both are business-critical, so contention becomes an architecture problem, not just a queueing problem

```
ETL ———————— Priority 1 ---\
Audit ————— Priority 1 ———\
Marketing ——— Priority 2 ———+——> [ WLM ] ——> +——————————————+
Finance ——— Priority 3 ———/                  | Redshift Cluster |
                                             +——————————————+
```

Why One Cluster Stops Working

Traffic patterns and isolation requirements keep growing:

- ETL prococess
- Marketing and business analytics
- Audit and compliance workloads with especially high sensitivity

Types of traffic

- Continuous baseline traffic sits on provisioned hardware all day
- But some workloads are bursty and should just finish as fast as possible, even if they only run once per day

Why One Cluster Stops Working

> A single shared cluster forces these very different workloads to fight over the same compute

The Goal - Isolated Clusters Per Workload

```
+——————————————————+    +——————————————————+
| ETL Cluster      |    | Audit Cluster    |
| Redshift         |    | Redshift         |
+——————————————————+    +——————————————————+

+——————————————————+    +——————————————————+
| Marketing Cluster|    | Finance Cluster  |
| Redshift         |    | Redshift         |
+——————————————————+    +——————————————————+
```

- The target state is workload isolation instead of one shared compute plane
- ETL, Audit, Marketing, and Finance each get their own Redshift cluster

More Than Just Performance

- Separate environments are often needed for testing:
  - pre-production validation
  - production-like testing
  - testing newer Redshift versions along the upgrade path

More Than Just Performance

- Redundancy also matters:
  - avoid a single point of failure
  - reduce blast radius during service outages
  - protect against version-specific bugs

More Than Just Performance

- Redshift Data Sharing wins:
  - Traffic isolation
  - Performance baseline per Redshift Cluster
  - Workloads can be isolated without duplicating storage
  - Internal billing
  - Redundancy
  - Decentralization

Redshift Architecture Crash Course

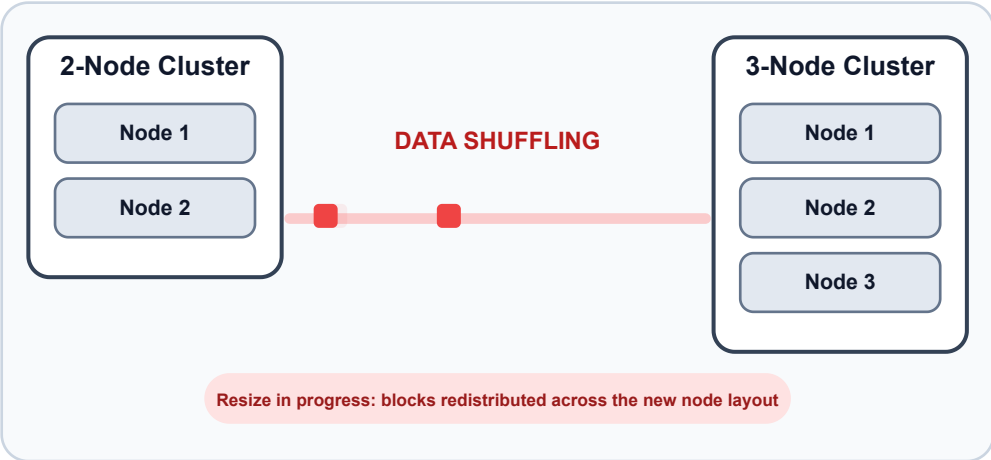
Pre-RA3 constraints, RA3 decoupling, and the path to Data Sharing

Pre-RA3 Architecture

```
+————————————————————————————————+
|         Single Cluster         |
|                                |
|     +————————————————————+     |
|     | Leader             |     |
|     |   Node             |     |
|     +————————————————————+     |
|        |          |            |
|        v          v            |
|   +————————+  +————————+       |
|   | Node 1 |  | Node 2 |       |
|   |        |  |        |       |
|   | Compute |  | Compute |      |
|   | Storage |  | Storage |      |
|   +————————+  +————————+       |
+————————————————————————————————+
```

- Compute and storage tightly coupled
- Painful resizing causing downtimes (data shuffling between nodes)

Pre-RA3 Resizing



- Pre-RA3 storage lived inside compute nodes
- Resizing meant redistributing data blocks across the new node layout
- This created long shuffle windows and operational disruption

RA3 Architecture

```
+──────────────────────────────────+
|     RA3 Compute Cluster          |
|                                  |
|  +─────────+                     |
|  | Leader  |                     |
|  |  Node   |                     |
|  +───+───+                       |
|      |                           |
|    +--+──────+                   |
|    |         |                   |
| +--v─────+ +--v─────+            |
| | Node 1 | | Node 2 |            |
| | Compute | | Compute |           |
| +─────────+ +─────────+            |
+──────────────+──────────────────+
               |
               v
+──────────────────────────────────+
| Redshift Managed Storage         |
| (S3 + cache + optimizations      |
+──────────────────────────────────+
```

- Compute and storage decoupled
- Independent scaling (elastic resize)

One Data Layer, Multiple Compute

```
+———————————————————+   +———————————————————+   +———————————————————+
| RA3 ETL Cluster   |   | RA3 BI Cluster    |   | Serverless for Audit |
|                   |   |                   |   |                   |
| +————————+        |   | +————————+        |   |  On-demand compute |
| | Leader |        |   | | Leader |        |   |  no fixed cluster |
| +———+————+        |   | +———+————+        |   |                   |
|     |             |   |     |             |   |                   |
|  +——+————+        |   |  +——+————+        |   |                   |
|  | Compute |      |   |  | Compute |      |   |                   |
|  +————————+        |   |  +————————+        |   |                   |
+——————————+————————+   +——————————+————————+   +——————————+————————+
           |                           |                           |
           v                           v                           v
     +———————————————————————————————————————————————————————————+
     | Redshift Managed Storage                                  |
     | (S3 + cache + optimizations)                              |
     +———————————————————————————————————————————————————————————+
```

- Same data
- No duplication
- Independent compute

Redshift Offerings

Provisioned

- Fixed clusters
- Predictable performance

Serverless

- Auto scaling
- Bursty / unpredictable workloads
- High priority use cases

Serverless Cost Model

- Billed per **RPU-seconds**, at least 60 :)
- ⚠️ Easy to overspend => Monitoring

Serverless Scaling

- RPU ~ 1 vCPU, 8 GB Memory ???
- Base RPUs, Max RPUs
- Intelligent Scaling in Amazon Redshift
  - Comples queries cost less and finish faster on properly scaled RPU
  - The goal is to avoid disk spills
  - AI adjusts over ~1–2 weeks

What Happens Under the Hood

- Metadata shared
- Data stays in S3
- Queries executed on consumer

👉 Zero-copy architecture

Pattern: Hub & Spoke

```
        +──────────────────────+
        |   ETL (Producer)     |
        +──────────+───────────+
                   |
                   v
        ═══════════════════════
               Data Share
        ═══════════════════════
          /        |         \
         v         v          v
     BI Team   Analytics   Reporting
```

- Producer cluster owns the data
- Exposes it through data shares

Pattern: Mesh

```
+————————+     +————————+
| Dept A |<——>| Dept B |
+——+——+     +——+——+
    |             |
    v             v
+————————+     +————————+
| Dept C |<——>| Dept D |
+————————+     +————————+
```

- Clusters have their own data
- Act both as Producers and Consumers

Pattern: Hybrid

- Mix of hub + mesh

👉 Reality

Lesson learned

Must match:

- Encryption settings
- Collation
- SUPER attribute settings
  - enable_case_sensitive_identifier
  - enable_case_sensitive_super_attribute

Helper objects

- UDFs

Schema naming across clusters

- sales.table1 on consumer
- sales.table1 on producer

Compatibility Issues

- Version mismatches (too much)

👉 Keep clusters aligned

Monitoring

- Know your baseline traffic
- RPUs consumed (Serverless)
- Health checks
  - Test Redshift availability
  - Test Datashare availability from consumers 👉 Heartbeat

Operations

There's no time for reverse engineering when shit hits the fan

- Script everything
- Document setup
- Automate deployment
- Runbooks
  - What to do when …

Final Thought

Stop fighting over one cluster.

Links

Jirka Hubacek

https://hubacek.xyz

GitHub Repository

ug-berlin-redshift-data-sharing